

Language Modeling with Transformers

Ayush Kumar Singh (mat. 266319)

University of Trento

ayushkumar.singh@studenti.unitn.it

1. Introduction

This report describes the two language modeling exercises, both trained and evaluated on Penn Treebank with perplexity as the metric [1]. In Part 1.A a decoder-only GPT-2 was implemented from scratch in PyTorch and trained as a word-level language model, then improved one component at a time from a deliberately weak baseline, covering the learning rate, model width, depth, attention heads, dropout, and weight tying. It reaches a test perplexity of 29.49. In Part 1.B the same pretrained GPT-2 was fine-tuned with a hand-written Low-Rank Adaptation (LoRA) [2] of the attention projections. Training only 0.35% of the parameters, it reaches a test perplexity of 19.05, well below the from-scratch model.

2. Implementation details

Part 1.A. The from-scratch model is a standard GPT-2 decoder, so the work on top of it is the tuning protocol and two regularizers. Starting from a small baseline ($d_{\text{model}}=20$, one head, one layer, feed-forward width 20), a suitable learning rate is found first, then each capacity hyperparameter (d_{model} , number of layers, heads, and feed-forward width) is changed one at a time and kept only when the development perplexity improves; unsuccessful changes are recorded as well. Widening the model and the feed-forward layer move the metric most, while extra depth and heads help only marginally. Dropout is then applied at four points: after the token and positional embedding sum, on the attention weights after the softmax, after the attention output projection, and after the final feed-forward layer. The dropout rate barely affects the floor (0.1, 0.3, and 0.5 land within a few tenths), but it stops the model from overfitting at the first epoch. Weight tying shares a single matrix between the token embedding and the output projection. Since these two matrices hold close to half of the parameters, tying them acts as a strong regularizer on a corpus of about a million tokens, and it gives the single largest improvement.

Part 1.B. The adapters are written by hand, without the PEFT library. Each block's attention is replaced with a custom module that loads the pretrained query, key, and value projections, freezes them, and adds a parallel low-rank update $\frac{\alpha}{r}BA$, with B initialized to zero so the adapter starts as a zero update and A drawn from a scaled normal. One detail is specific to the Hugging Face loader: `from_pretrained` builds the model inside a `no_init_weights` context that turns every `torch.nn.init` call into a no-op, so A must be assigned directly rather than through `nn.init`; otherwise it stays uninitialized and the first step produces a NaN loss. The scaling is kept proportional to the rank ($\alpha = 2r$) so the sweep isolates capacity, and only the adapters are trained, 884K of the 124M parameters.

3. Results

Both parts are evaluated by perplexity on the Penn Treebank test set, computed as the exponential of the mean per-token negative log-likelihood. Each configuration is trained for up to 100 epochs and the checkpoint with the lowest development perplexity is kept, selected by early stopping with patience 3 on a held-out split, so the reported test number always corresponds to the saved weights.

Figure 1 and Table 1 track the incremental tuning of Part 1.A. The perplexity falls steeply over the first capacity changes, where widening the model and the feed-forward layer account for most of the gain, and then flattens as further depth and heads add little. The final and largest drop comes from weight tying, which is the only change that pushes the development minimum below 40; dropout, although its rate barely affects the floor, is what lets the model keep improving past the first epoch instead of overfitting immediately.

Figure 2 and Table 2 report the LoRA rank sweep against the from-scratch model. Perplexity falls as the rank grows, with the gain per doubling shrinking from about 1.1 to 0.6, and the saved $r=16$ adapter reaches 19.05. This is more than 10 perplexity points below the 26.8M-parameter from-scratch model and is obtained while training only 884K parameters, which indicates how much the pretrained weights contribute beyond what the small model can learn on a corpus of this size.

Taken together, the two parts isolate different sources of the perplexity reduction. Part 1.A shows how far a small model trained from scratch can be pushed by capacity and regularization alone, with weight tying as the decisive lever, while Part 1.B shows that even the smallest adapter, at rank 4 with 221K trainable parameters, already beats the best from-scratch model by almost 9 perplexity points. Most of the remaining gap is therefore closed by the pretrained representation rather than by the size of the adaptation.

Reported numbers come from single runs. Repeating the same configuration shifts the result by a few tenths of a perplexity point because of CUDA reduction nondeterminism, so each value is read from the run whose weights were saved to the checkpoint.

Use of AI tools. Claude was used minimally, for debugging and refinement.

4. References

- [1] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed., 2024, draft, <https://web.stanford.edu/~jurafsky/slp3/>.
- [2] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations (ICLR)*, 2022.

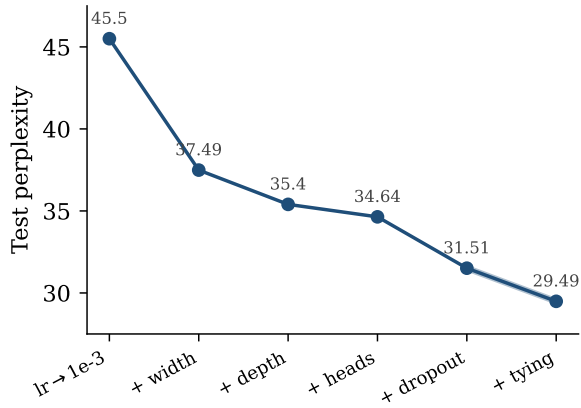


Figure 1: *Part 1.A incremental tuning. Test perplexity after each lever is added on top of the previous configuration; weight tying gives the largest single drop.*

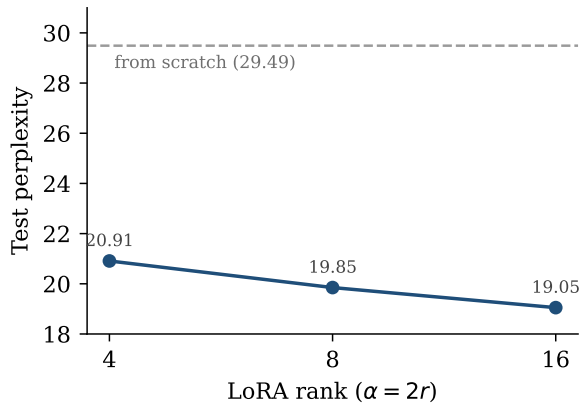


Figure 2: *Part 1.B LoRA rank sweep. Perplexity falls with rank and stays well below the from-scratch model (dashed line).*

Table 1: *Part 1.A: incremental tuning on Penn Treebank. Each row adds one change on top of the previous configuration.*

Change	Test PPL
Learning rate $\rightarrow 10^{-3}$	45.50
+ width ($d_{\text{model}}=256$)	37.49
+ depth (4 layers)	35.40
+ heads (8)	34.64
+ dropout (0.3)	31.51
+ weight tying	29.49

Table 2: *Part 1.B: LoRA rank sweep with $\alpha = 2r$, compared to the from-scratch model. The $r=16$ row is the saved checkpoint.*

Model	Trainable	Test PPL
From scratch (1.A)	26.8M	29.49
LoRA $r=4$	221K	20.91
LoRA $r=8$	442K	19.85
LoRA $r=16$	884K	19.05